



A Self-Testing Framework for Verification and Validation of a RISC-V-Based System with a Co-processor

Sven Gesper¹ · Fabian Stuckmann¹ · Lucy Wöbbekind¹ · Guillermo Payá-Vayá¹

Received: 26 April 2024 / Accepted: 4 April 2026
© The Author(s) 2026

Abstract

Compliance testing is mandatory when implementing the hardware architecture of a specific instruction set. The official compliance test suite with handwritten test cases for RISC-V can be helpful for this task. However, a high-quality test suite requires significant manual effort and cannot easily adapt to specific processor hardware architecture organization implementation aspects such as single-cycle, multi-cycle, or pipeline (with a different number of pipeline stages) configurations, or include an additional co-processor. This paper uses the PATARA framework, based on the REVERSI approach, to generate randomized, self-testing test cases for any RISC-V hardware implementation. The REVERSI method verifies the functionality within the same test program without requiring a golden reference model (e.g., simulator) and speeds up post-silicon validation times. With extensions to cover all possible hardware architecture implementation hazards and cache misses, a 6 pipeline-stages RV32IM hardware architecture implementation is verified functionally, reaching up to 100 % condition coverage with the REVERSI self-testing approach against 78.94% coverage achieved by the official handwritten compliance test framework. Moreover, the self-testing framework is extended in this paper to verify a vertical vector co-processor. Similar to the REVERSI method, two calculation paths for instruction are created to verify the execution. The constraint random generated vector instructions get validated by a software twin, executed on the RISC-V processor. The code coverage of the co-processor reaches up to 100 % condition coverage with self-verification on the system.

Keywords RISC-V · Co-processor · Functional verification · Code coverage · REVERSI

✉ Sven Gesper
s.gesper@tu-braunschweig.de

¹ Chip Design for Embedded Computing, Technische Universität Braunschweig, Braunschweig, Germany

1 Introduction

The RISC-V instruction set architecture (ISA) [33–35] is an open and free accessible specification that has been widely used in academia and industry in recent years. Many hardware implementations with support for various features and micro-architecture optimization aspects have emerged as a result of open-source availability. The differences between those hardware implementations include, for example, the pipeline depths. Also, some ISA extensions, like multiplication or division, can be implemented using multi-cycle approaches to keep the operating frequency high. Moreover, the control mechanism for handling hazards, introduced by pipeline dependencies and conflicts, may implement different strategies for resolving them. These hardware implementation differences and the explicit support of domain-specific instruction set extensions make the use of a generic verification framework challenging. It is worth mentioning that an alternative of domain-specific instruction set extensions to increase the processing performance is the combination with a dedicated hardware accelerator, also called co-processor. Due to the increased complexity of coupled systems, verification becomes even more challenging.

The goal of functional verification is to identify and localize bugs in the design that cause the design to behave incorrectly. Normally, this functional verification is performed twice, during a pre-silicon verification phase before the fabrication of the final chip in the foundry and during a post-silicon validation phase directly on the fabricated chip. The overall functional verification process is becoming more difficult, and the engineering resources to guarantee a functionally correct design are increasing [18].

In general terms, any verification framework for any processor implementation can be divided into two parts. (a) The input stimulus, i.e., the set of test programs to be executed on the target processor implementation. For example, these test programs can be automatically generated or specifically handwritten. And (b) verifying the correct execution, i.e., how to validate the correct execution of the test program. For example, this validation can be performed using a given output reference or a simulator.

This paper extends the previously published paper [10] of a verification and validation framework, called PATARA.¹ The verification framework is used to verify any custom RISC-V hardware architecture implementation. The self-testing verification capabilities are extended with a new twin-based software approach for verifying programmable-based co-processor architectures, like complex vector co-processor arrays. The PATARA framework is based on the REVERSI approach [31], which is used for the automatic generation of randomized self-test programs to detect hardware errors without requiring a golden model for the validation of the functional correctness of the executed test programs. The proposed extensions improve the generated random self-test programs with the scope for verifying all kinds of pipeline hazards and cache miss situations derived from the custom RISC-V hardware architecture implementation. Furthermore, the framework for verifying programmable co-

¹The open-source PATARA framework [29], including the RISC-V extensions, is available on GitHub <https://github.com/tubs-eis/PATARA>

processor architectures extends the self-testing method by generating test programs that execute the same computation on two different paths, i.e., on the co-processor itself and the RISC-V processor. This *twin-based approach* allows on-chip pre-silicon verification, achieving full code coverage. For later post-silicon validation, the presented framework enables self-testing with low communication overheads. Overall, this paper presents the following contributions:

- A new approach for creating randomized self-test programs to automatically generate any kind of data, control, and structural hazards for the RISC-V ISA.
- A mechanism to generate data and instruction cache misses, which can be configured depending on the implemented cache memories, e.g., the size of the cache line.
- A twin-based methodology to verify any programmable co-processor architecture, demonstrated on a vertical vector co-processor, called V²PRO.
- An evaluation of the quality of the generated instruction test programs, using code coverage metrics, and a comparison to the official handwritten compliance test framework for RISC-V.

This paper is organized as follows: In Sect. 2, an overview of related verification methods is given. The verified system, containing a RISC-V architecture implementation, and the V²PRO co-processor are briefly described in Sect. 3. The REVERSI verification approach of the RISC-V verification framework PATARA is explained in Sect. 4. This includes several extensions of the framework to improve the quality of the generated test programs for generic implementations (e.g., pipeline configurations). Moreover, the twin-based verification method for the co-processor is also introduced. The proposed system verification framework is evaluated by means of a hardware simulation and code coverage metrics in Sect. 5. The extensions made to the PATARA framework to ensure complete code coverage of the RV32IM implementation, and their results are discussed in Sect. 5.1, while the co-processor verification is discussed in Sect. 5.2. Finally, the conclusion of the verification framework is drawn in Sect. 6.

2 Related Work

2.1 RISC-V-Based Processor Verification

Pre-silicon verification techniques compare the behavior of an architecture implementation to the specification. The reference behavior for the RISC-V ISA can be generated with a so-called golden model that can be an Instruction Set Simulator (ISS), such as the widely used Imperas OVPSim model [17]. The simulators may not include external components (e.g., memory, buses), which limits the test capabilities. Here, virtual prototypes extend the test's scope by simulating all system components, such as (virtual) memories, caches, and other peripherals [13]. The use of such complex systems can increase the level of test coverage and allow the testing of advanced

RISC-V architectures. A disadvantage of increasing the complexity of the reference simulator system is that new sources of error are introduced. This was shown in [20].

In terms of functional verification, errors in the design are targeted to be triggered and detected by a set of test applications. The quality of verification procedures can be measured through code coverage metrics that quantify how much of the design (e.g., VHDL code) is covered during the execution of test applications. The goal of maximizing the code coverage can be implemented with automatic test generators, as demonstrated in [15]. However, these increase the number of test sequences that have to be executed. With the increasing complexity of system designs, the testing methodology requires even more complex test cases. A verification strategy is required for a fast and complete verification with the lowest possible number of tests executed. This can be achieved with manually optimized test cases but lacks scale for various hardware architecture implementations.

Another stage during processor development is the validation of post-silicon cores. The goal of this validation is not only the detection of design errors [32]), but also includes fabrication faults, electrical errors, or datapath-specific timing violations. Due to the limited observability of post-silicon hardware, it becomes difficult to monitor internal signals to reproduce bugs [22]). On an industrial scale, validation (and testing) of shipped-out processors is required to verify the correctness of a system.

For the RISC-V instruction set, there exists an official RISC-V unit [25] and compliance [26] test suite with handwritten test cases, claiming to reach nearly 100% functional coverage as stated in [16]. But the comprehensiveness of these test suites is still limited, regarding implementation-dependent hardware architecture coverage.

In addition to handwritten test cases, automatic test generators can be improved by using predefined randomized test sequences focused on valid instruction streams (positive testing). An example of a verification framework for randomized RISC-V instruction streams is Google's RISC-V DV [1, 2]. This framework generates continuous instruction streams and uses a reference simulator to verify execution traces.

Another approach is to use optimization strategies to guide the generation of test programs. In [28], an evolutionary algorithm is used to modify the executed instructions, increasing the coverage by use of a fitness function for coverage. This method achieved up to 90% code coverage for an exemplary RV32IMFC architecture (RI5CY [9]). Another improvement is the use of unconstrained randomized test sequence generation [7]. The proposed method generates sequences based on a formal description of the instruction set and selects instructions based on a solver called Satisfiability Modulo Theories (SMT). Another strategy is the fuzzing-based test suite generation approach, which is presented in [14]. In addition to the positive testing-focused methods, this approach focuses on negative testing (e.g., illegal instructions, and exception handling). For this method, any sequence is generated, including invalid instructions, and the architecture implementation is expected to identify and handle invalid instructions accordingly.

The verification of the correctness of such test cases requires comparing the execution of a golden reference model, which increases the verification time. Moreover, when used for post-silicon validation (e.g. scaled, on a large number of devices), these verification methods are limited by the duration of the reference generation and the communication to a host system comparing the test results for the tested device.

2.2 Co-Processor Architecture Verification

With online validation on a device, multi-core systems may detect failures by a redundant execution on its cores. Also, software checkers can identify core failures during idle phases [19]. Adding specialized hardware [4] to verify critical instructions or specific tasks by a reduced in-order reference calculation unit (e.g., for control, compute, or memory tasks) gives a system the possibility of self-testing. Using this approach of on-chip validation, the system can detect failures as defined for post-silicon validation.

One way to verify the functionality of co-processors is to execute specific test cases. In a recent study [23], different crypto-processors designed for AES, ECC, SHA, and RNG were verified using a variety of specific software function tests. The tests aimed to test the functionality of the crypto-processors thoroughly. The verification framework used in the study [36] builds and executes the tests in both simulation and FPGA emulation environments.

The utilization of formal verification for co-processors can be advantageous [27] but has significant drawbacks such as expensive manual steps required, or complexity limitations [8].

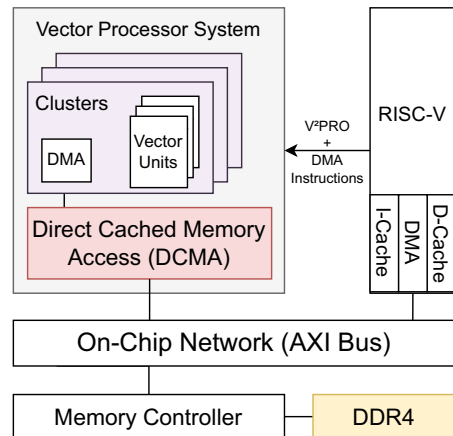
The verification of a floating-point calculation unit requires datapath verification. While the *IEEE test suites* are commonly used for this task, they fall short in providing coverage for certain hardware implementations. To improve coverage, generating random data based on constraints can be highly effective, as demonstrated by [8]. The verification process involves comparing the calculated results to the expected results derived from an offline reference model.

This paper extends the REVERSI approach [31] to generate randomized self-testing programs for RISC-V hardware implementations to circumvent limitations regarding reference generation duration. Moreover, a vector co-processor tightly coupled to a RISC-V processor is validated by executing vector instructions on the verified in-order RISC-V core through a twin-based approach. This verification process eliminates the need for previously computing a golden reference and the necessary data transfers since the RISC-V processor is used for this purpose.

3 Co-Processor with RISC-V-Based System

The considered architecture for the verification task features a high-performance co-processor, named V²PRO (Vertical Vector Processor) [24, 30], which is programmable and consists of multiple parallel processing vector lanes with individual register files. The structure is shown in Fig. 1. The system includes the V²PRO (top left), the interconnect to an external memory (bottom), and a RISC-V core. The memory access ports of the RISC-V are cached and share the AXI (Advanced eXtensible Interface) interconnection with the vector co-processor to access the external memory. Multiple DMA (Direct Memory Access) units from the vector co-processor access the external memory via a shared DCMA (Direct Cached Memory Access) unit. This DCMA features a multi-port cache system, allowing parallel access [30].

Fig. 1 System Overview of the RISC-V with attached vector co-processor, including the access of multiple DMA units to an external memory via a shared cache (DCMA)



The focus of this paper is the verification methodology used for the RISC-V core and the vector co-processor. The components of the external memory access are omitted and require external testing (e.g., by AXI verification IP [3], methods of DDR Controller verification [21], or DMA verification [11, 12]). Therefore, in the later verification, the external DDR memory, DMA units, memory caches, and the AXI interconnect are excluded from the verification and assumed to work functionally correctly.

Generally speaking, the RISC-V processor performs all control flow tasks, including the generation of vector instructions for the vector co-processor. Therefore, the vector co-processor is in charge of executing the vector instructions provided by the RISC-V and based on a sequential vectorization paradigm. This vectorization paradigm allows complex addressing forms for vector elements, as the execution of a long vector is calculated elementwise, sequentially (more information is given in [30]). The co-processor uses parallel vector lanes for performance scale purposes.

To send instructions to the vector co-processor, the RISC-V memory access instructions are used, and dedicated memory addresses are mapped to the vector processor control interface. The generation of the co-processor's vector instructions by the RISC-V core consists of logical, base arithmetic, and some control flow instructions. Therefore, the RV32IM ISA was chosen as a processor's ISA.

3.1 The RISC-V RV32IM ISA

The RISC-V instruction set consists of 45 base instructions (RV32I) [33–35], including computational, control flow, system, and memory transfer instructions. Moreover, several extensions of the instruction set can be used to further extend the functionality, like, for example, the *M* extension, which specifies 8 additional instructions for multiplication and division operations. Other extensions add support for floating point operation (F/D/Q), atomic instructions (A), or compressed 16-bit instructions for smaller instruction memory footprints (C). Furthermore, the instruction set allows the implementation of custom extensions for specific operations by leaving some regions of the instruction encoding space open for customization. The instruc-

Table 1 RISC-V Instruction Set for RV32I and M Extension, used for verification in this paper

Computational		Memory		Control		RV32M	
Instruction	Type	Instruction	Type	Instruction	Type	instruction	type
add	R	lb	I	beq	SB	mul	R
sub	R	lbu	I	bne	SB	mulh	R
sll	R	lh	I	blt	SB	mulhu	R
srl	R	lhu	I	bltu	SB	mulhsu	R
sra	R	lw	I	bge	SB	div	R
and	R	sb	S	bgeu	SB	divu	R
or	R	sh	S	jal	UJ	rem	R
xor	R	sw	S	jalr	I	remu	R
slt	R						
sltu	R						
addi	I						
slli	I						
srli	I						
srai	I						
andi	I						
ori	I						
xori	I						
slti	I						
sltiu	I						
lui	U						
auipc	U						

Table 2 RISC-V instruction types

31	30	25	24	21	20	19	15	14	12	11	8	7	6	0	
funct7			rs2			rs1		funct3		rd			opcode		R-type
imm[11:0]						rs1		funct3		rd			opcode		I-type
imm[11:5]			rs2			rs1		funct3		imm[4:0]			opcode		S-type
imm[12]	imm[10:5]					rs1		funct3		imm[4:1]	imm[11]		opcode		B-type
imm[31:12]										rd			opcode		U-type
imm[20]	imm[10:1]				imm[11]	imm[19:12]				rd			opcode		J-type

tion set of RV32IM, used in this paper, is shown in Table 1. Besides computational instructions, control transfer instructions, system instructions, and memory access instructions are defined by the instruction set. The *M* extension adds 8 additional instructions for multiplication and division operations.

Depending on the opcode, the instruction type can use register or immediate values. The base RV32I architecture defines 32 registers (including the zero register \$0). Immediate values are encoded in corresponding instruction fields. The different instruction format types are shown in Table 2.

The hardware architecture implementation of RV32IM used in this paper for the evaluation of the extended PATARA framework is shown in Fig. 2. The hardware architecture implements a pipeline with 6 stages; Instruction Fetch (IF), Instruction Decode (ID), Execution (EX), Memory Access (MEM 1 & MEM 2), and Write Back (WB). Data hazards are resolved by forwarding mechanisms between the pipeline

stages. In case of a load data hazard, stall cycles are introduced by a hazard detection unit located within the ID stage until forwarding can resolve it. The multiplication unit is divided into two stages (EX & MEM1) and introduces a data hazard, which causes the pipeline to stall for one cycle to resolve it. Data cache misses stall the pipeline as the in-order execution requires the memory access to finish. The implementation of the division unit uses a multi-cycle approach, which focuses on a small area footprint for embedded systems and requires to stall the pipeline until a division operation is completed. Branch prediction is done statically, assuming that the branches are always not taken. If a branch is taken, the pipeline is flushed. Since the branch condition is evaluated in the EX stage, two instructions have to be flushed if the branch is taken. However, unconditional jump instructions only flush one invalid instruction, due to their execution in the ID stage. The access to control and status registers is executed in the ID and EX stage. As some of the system instructions require modification of the program counter, stall conditions are introduced to execute these instructions.

3.2 V²PRO Co-Processor

The execution of compute-intense tasks can be done by the co-processor of the RISC-V core. The vector co-processor executes vertical vector instructions to allow the SIMD concept to process parallel data within a single vector instruction. With instruction broadcast to an array of parallel vector lanes, the vertical (sequential) execution of the processing vector lanes achieves a high parallel processing performance. The instruction set of the vector co-processor includes arithmetical, logical, data movement, and some custom operations. The vector ISA is shown in Table 3. Each vector instruction uses two input and one output operand (three-operand-machine). The address of the individual operands is calculated during the processing of the vector elements and allows vector elements to be addressed flexibly. A complex address calculation provides three-dimensional addressing capabilities. With this, each operand contains an offset, the factors α (multiplied with x), β (multiplied with y), and γ (multiplied with z). The length of the processed vector is defined by the limits of the

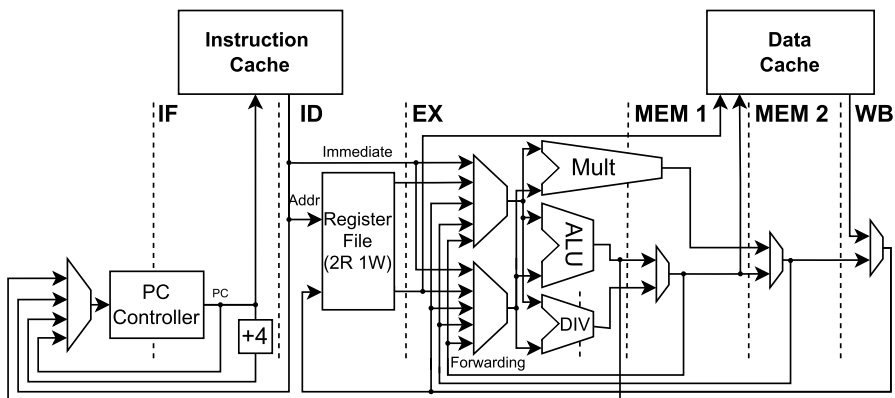


Fig. 2 Pipeline structure of the RISC-V implementation according to RV32IM

Table 3 Supported instruction set architecture of the vector co-processor

Computational		Memory		Other	
Instruction	Operation	Instruc- tion	Operation	in- struc- tion	operation
ADD	DST = SRC1 + SRC2	LOAD	$\text{chain}_o = \text{LM}[\text{SRC1} + \text{SRC2}_{\text{IMM}}]_{16\text{-bit}}$	MIN	DST = Min(SRC1, SRC2)
SUB	DST = SRC1 - SRC2	LOADS	LOAD, sign-extended	MAX	DST = Max(SRC1, SRC2)
MULL	$\text{accu} = \text{SRC1} * \text{SRC2}$, DST = $\text{accu}(\text{low bits})$	LOADB	$\text{chain}_o = \text{LM}[\text{SRC1} + \text{SRC2}_{\text{IMM}}]_{8\text{-bit}}$	ABS	DST = Abs(SRC1)
MULH ¹	$\text{accu} = \text{SRC1} * \text{SRC2}$, DST = $\text{accu}(\text{high bits})$	LOADBS	LOADB, sign-extended		
MACL ²	$\text{accu} = \text{accu} + \text{SRC1} * \text{SRC2}$, DST = $\text{accu}(\text{low bits})$	STORE	$\text{LM}[\text{DST} + \text{SRC2}_{\text{IMM}}]_{16\text{-bit}} = \text{chain}_i$		
MACH ^{2,3}	$\text{accu} = \text{accu} + \text{SRC1} * \text{SRC2}$, DST = $\text{accu}(\text{high bits})$				
MACL/H_PRE ³	$\text{accu}_{\text{zero-init}}$, then MACL/H				
SLL	DST = SRC1 < SRC2 (logical)				
SRL	DST = SRC1 > SRC2 (logical)				
SRA	DST = SRC1 > SRC2 (arithmetic)				
Conditional		Logic			
Instruction	Operation	Instruction	Operation		
MV_PL	(move) DST = SRC2, when SRC1 >= 0, else no-operation	XOR	DST = SRC1 $\oplus$ SRC2		

Table 3 Continued

Conditional		Logic	
Instruction	Operation	Instruction	Operation
MV_MI	(move) DST = SRC2, when SRC1 < 0, else no-operation	XNOR	$DST = SRC1 \oplus \overline{SRC2}$
MV_ZE	(move) DST = SRC2, when SRC1 == 0, else no-operation	AND	DST = SRC1 & SRC2
MV_NZ	(move) DST = SRC2, when SRC1 != 0, else no-operation	NAND	$DST = \overline{SRC1 \& SRC2}$
MULL_NEG	DST = MULL, when SRC1 < 0, else SRC1	OR	DST = SRC1 SRC2
MULH_NEG ¹	DST = MULH, when SRC1 < 0, else SRC1	NOR	$DST = \overline{SRC1 SRC2}$
MULL_POS	DST = MULL, when SRC1 >= 0, else SRC1		
MULH_POS ¹	DST = MULH, when SRC1 >= 0, else SRC1		
SRA_NEG	DST = SRA, when SRC1 < 0, else SRC1		
SRA_POS	DST = SRA, when SRC1 >= 0, else SRC1		

¹ MULH: "high" position is configurable by a special configuration register

² MACL/H: accumulator reset mode & value are configurable by special configuration registers

³ MACH: "high" position is configurable by a special configuration register

Each vector instruction is evaluated on every vector element (three operands). The vector co-processor's data path width is 24-bit. The accumulation register, used in multiplication instructions, is a single 48-bit register

running variables x , y , and z (x_{end} , y_{end} , z_{end}). The operand addressing calculation is defined by

$$Address = x * \alpha + y * \beta + z * \gamma + \delta \quad (1)$$

and is implemented for the three operands in the V²PRO individually.

As mentioned, vector lanes process the vector elements sequentially. The implementation of the hardware is pipelined, and through high frequency, high throughput is reached. During the pipelined execution of a vector, neighbor vector lanes can exchange the processed vector data by a chaining mechanism (see Fig. 3). The selection of the vector operands source is encoded in the vector instructions and includes

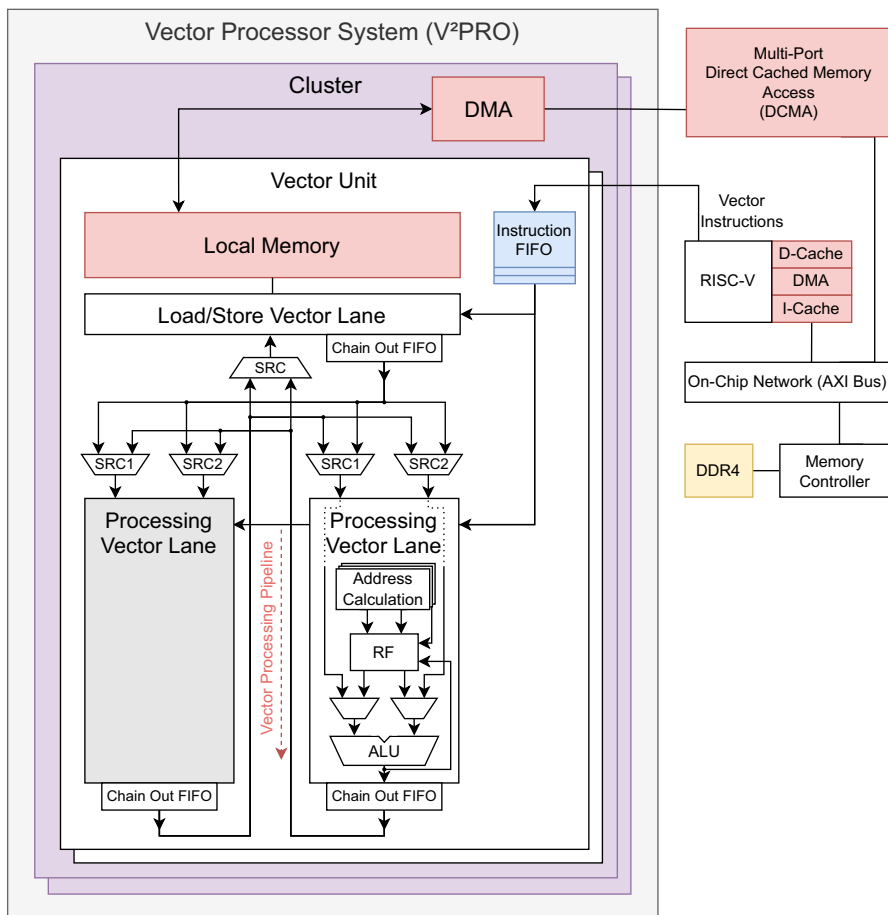


Fig. 3 Schematic representation of the V²PRO, attached to the RISC-V, illustrating the architecture's vector lane chaining network and the access of DMA units to external DDR4 memory through a shared cache (DCMA)

the selection of data from a chained source, from the vector lane's register file, or by immediate values.

For increased processing capabilities, the co-processor array can be scaled by its number of parallel units and clusters. Each vector unit features two parallel executing processing vector lanes, a local memory, and a load-store vector lane for memory access to the local memory. Inside a cluster, a DMA unit transfers data between the local memories in the vector units and the external memory (cached access to the AXI interconnect by the DCMA). The vector instructions for execution in the vector lanes are buffered in the vector units in a first-in-first-out instruction memory (FIFO) to uncouple the vector execution from the instruction issue of the RISC-V processor and to enable seamless execution of vector instructions (see blue FIFO in Fig 3). A filled instruction FIFO, targeted by a new vector instruction, stalls the generation of vector instructions until the memory quits its full state.

```

1 // (1) Modification Operation(s)
2 add t2, t1, t0 // t2 ← Target register
3              // t1 ← Focus register
4              // t0 ← Random register
5
6 // (2) Restoring Operation(s)
7 sub t2, t2, t0 // Restoring operation
8
9 // (3) Compare Focus and Target register
10 beq t2, t1, SUCCESS // evaluate t2 == t1

```

Fig. 4 Assembler code for an addition *test-instruction*. REVERSI performs a *modification operation* (add) on a focus value and random data. The result of the *restoring operation* (sub) is compared to the initial focus value to detect errors

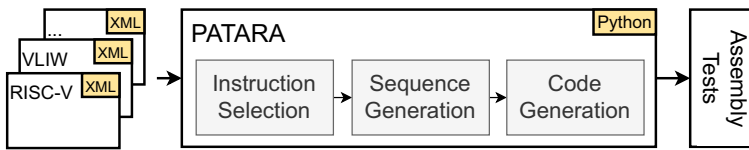


Fig. 5 Overview of the PATARA framework. Different instruction sets and processors can be configured with configuration files, and PATARA framework automatically generates custom self-test assembly programs

4 Proposed Verification and Validation Framework

For pre-silicon verification and post-silicon validation of the system with a RISC-V core and the V²PRO co-processor, the REVERSI approach is used on the RISC-V core, and the RISC-V is then used to validate the V²PRO co-processor, using a twin-based verification approach on the coupled system itself. After the random generation of vector instructions and processed data, the execution is performed on both twins (V²PRO and RISC-V twin). The result of the V²PRO execution is compared to the reference calculation on the (already verified) RISC-V twin, to detect failures of the processing on the co-processor. Instead of using an offline reference model, the RISC-V processor is used as the golden reference model. To uncouple the verification further from communication bottlenecks, the test instructions and the processed data are generated by an online constrained random process on the RISC-V. With this method, the communication of the system with an external test unit is minimized.

4.1 RISC-V Verification with the REVERSI Approach

The REVERSI approach is a post-silicon validation technique for generating randomized self-test programs for any fabricated processor architecture. The general structure of a self-test program is shown in Fig. 4. First, initial data is changed with a *modification operation*, which contains the test instruction. Then, the initial state is restored with a *restoring operation*. The final part of the REVERSI method consists of a comparison of the initial value with the result of the *restoring operation*. If these are equal, an error-free execution occurs. The execution of such self-test programs

during the pre-silicon verification and/or post-silicon validation process allows verification of the functional correct of the target processor by itself.

REVERSI requires that a test instruction of the ISA implements a *modification* and *restoring* operation. The test instruction should only be used in the *modification operation* so that a systematic error in the instruction does not mask an error that occurs inside the *modification* and *restoring operation*. The *modification operation* has a *focus register* and *random register* as source operands and stores the result in a *target register*. The *restoring operation* uses the *target register* and the *random register* to restore the value of the focus register. The result of the *restoring operation* is saved in the target register. Thus, in the comparison stage, the *focus* and *target register* are compared to ensure that the execution was error-free.

Each instruction of the ISA is implemented by defining *modification* and *restoring operations*. For some instructions, such as the shift left operation, some information, which is required for the reversal or restoration, can be lost during the *modification operation*. Thus, control code is added to the test instruction in the *modification* and *restoring operations* to enable the full restorability of the *focus register*.

To increase the complexity of the self-test programs, a stack consisting of all instructions of the ISA can be interleaved. In this interleaving process, *modification operations* of test cases are chained together, i.e., the result of a *modification operation* is the input to the next *modification operation* of the following test case. To restore all the test cases, the *restoring operations* are performed in the reverse order in which the modifications were performed. To further increase test complexity, multiple stacks can be interleaved. Therefore, arbitrarily long test cases can be generated.

4.2 The Open-Source PATARA Framework

PATARA [29] is an open-source tool that implements and extends the REVERSI approach [31]. The overview of the PATARA framework is depicted in Fig. 5. PATARA imports the processor and instruction descriptions that are provided with XML description files. These can be changed for different processors and instruction sets. In the first step, PATARA selects from the XML file the list of instructions to be tested. Then, different instruction sequences are generated depending on the test mode. These can be single-instruction tests or more complex test procedures. The final step generates the *modification* and *restoring operations* and the comparison

```

1 <add>
2   <instr>add TargetReg, FocusReg, randValue</instr>
3   <sequence-instr>add PlainTargetReg, FocusSequenceReg,
4     randValue</sequence-instr>
5   <reverse>sub TargetReg, TargetReg, randValue</reverse>
6
7   <signage>signed</signage>
8   <immediate>None/i12</immediate>
9   <type>R</type>
10  ...
11 </add>

```

Fig. 6 XML Definition of the ADD instruction with its reverse SUB instruction

code. Additionally, the content of the register file is initialized with random values. The output of PATARA is a set of assembly files for the target processor.

For instruction set extensions (such as custom ISA extensions in RISC-V), the description files require the layout of the new instructions and the corresponding sequence to restore custom operations. Depending on the complexity of the custom operations, the *restoring operation* includes multiple instructions to restore the original data. For higher-level implementations of restore procedures, algorithmic functions can be called in the reversing procedure. In Fig. 6, the PATARA framework definition of the RISC-V ADD instruction is shown. To generate instruction sequences (as explained in the following section) the sequence instruction definition is used in line 3. The reverse counterpart and variants of immediate and signage data are configured as shown in line 5. The further definitions (line 7 and following) allow custom instruction-specific variants that create derivatives of the ADD instruction.

4.3 RISC-V Extensions Implemented in PATARA Framework

In this paper, processor and instruction definitions for RISC-V were added to the PATARA framework [29]. The instruction descriptions include the full *RV32IM* instruction set, including arithmetic, logic, and control flow instructions, as shown in Table 1. Furthermore, the PATARA framework was extended with hardware implementation-dependent pipeline hazard generation mechanisms, and a new approach to generate instruction and data cache misses.

The **first RISC-V specific extension** enables the use of different immediate types in the instruction set. An example is the *shamt immediate value* used in the shift instruction, which only requires 5 bits compared to the regular 12 bits of I-type instructions (see Table 2). To incorporate the variable immediate sizes, the instruction description encodes the immediate length for each instruction in the instruction XML. When generating randomized immediate data, PATARA respects the immediate lengths during test-program generation.

The **second RISC-V specific extension** is concerned with the length of interleaving sequences due to the limited destination target address range of ($\pm 4KiB$) for the conditional branch instructions. Some instruction tests, such as branch on equal, contain conditional branches in the *modification operation*, while the corresponding branch target is inside the *restoring operation*. These tests are *branch-limiting tests* as there is a limit on how many instructions can be placed between the *modification* and *restoring operation* of these tests.

After a branch-limiting test is detected in the stack of interleaving instructions, the assembly lines are counted for each subsequent test case. If the branch limit is reached, the stack is split into two sub-stacks. The first sub-stack is executed normally, i.e., *modification* and *restoring operations* are generated. Thus, the result of the first sub-stack should match the initial data. The result of the first sub-stack is then passed to the second sub-stack. The process of detecting branch-limiting tests is repeated for the second sub-stack until all instructions have been divided into sub-stacks. Thus, an arbitrary number of sub-stacks are created, and long sequences of interleaved instructions can be tested. The length of the interleaving tests can be configured in the PATARA framework.

For the fully functional verification of the **pipeline hazard detection mechanism**, PATARA was extended to generate arbitrary fixed-length sequences of instructions to test the different pipeline hazards, covering all possible forwarding path selections (see source multiplexers of EX stage in Fig. 2) and the stall mechanisms. With these *hazard-triggering* instruction sequences, the generated test cases include the evaluation of hazard conditions of any hardware pipeline configuration. The instruction sequences for these hazard coverage are automatically generated based on all permutations of the instruction types of the ISA and include all possible data dependency (i.e., data forwarding) combinations. The required sequence length depends on the number of pipeline stages (e.g., for a 6 pipeline stages implementation, a sequence of 4 consecutive instructions is needed for full coverage). The instructions for a sequence are randomly selected based on their instruction type. In order to generate these sequences, this extension takes advantage of the regular interleaving approach (explained in Sect. 4.1), where the initial data is forwarded from one test instruction to the next one. Additionally, random filler instructions can be inserted into the generated sequence to test the forwarding mechanisms. The introduced instruction does not interfere with the data-dependency of the test sequence and enables to cover further hazard state switches as shown in Fig. 9.

Another extension in PATARA to validate **pipeline hazard detection mechanisms** is the full use of the data forwarding paths, i.e., to check all possible combinations using both source operands. In a regular test case, the *focus register* is always the first source operand, while the second source operand supplies the random data. The second operand is either a register or can be an immediate value. Thus, the forwarding hazard detection fails to test the forwarding path to the second source operand. To resolve this issue, the positions of the focus and random values are swapped so that forwarding hazard detection on both source operands is tested. The probability of swapping operands during the test case generation can be configured in the PATARA framework.

Finally, a methodology for testing the **instruction and data caches** is also introduced. For the data cache (dcache), the data address generation is modified to introduce cache line misses based on the cache line width specified in the target processor description. Instruction cache (icache) tests are limited to interleaving test sequences, which provoke icache misses for the provided cache configuration described in the target processor description. The icache miss test consists of a jump instruction, filler

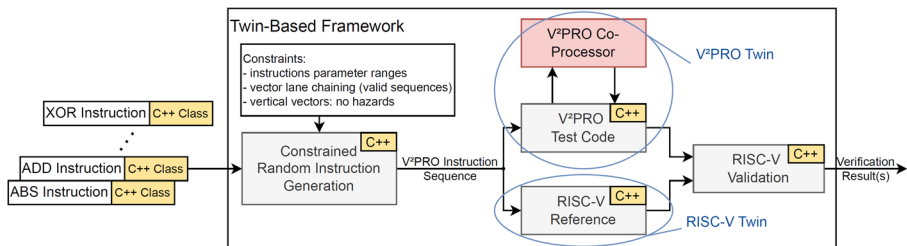


Fig. 7 Twin-based verification framework for co-processor instruction generation. Constraints are used to select random vertical vector instructions. The co-processor is executed and validated by the RISC-V and its reference execution on its twin

instructions, and a jump destination. The filler instruction is repeated to fill the cache line, thus the jump destination causes a cache miss.

4.4 V²PRO Verification Using a Twin-Based Approach

The following describes the method for verification of the V²PRO co-processor. The RISC-V system does the verification of executed vector instructions. In order to achieve this, a simulation model runs on the RISC-V processor and calculates the reference result for the vector co-processor execution. There are two paths (twins) for the processing of vector instructions:

1. Calculation of the vector instruction on the co-processor hardware.
2. Emulation of the execution on the verified RISC-V by a software twin.

Both processors use external memory with cached access, requiring cache synchronization for access to data sections from the other processor. For both execution paths, the same input data are used. The twin-based verification process is depicted in Fig. 7, beginning with the random vector instruction generation, execution on the two twins, and the verification of the results.

The application, running on the RISC-V, may generate test sequences of vector instructions or use pre-defined sets of random vector instructions. In both cases, the generation process for random vector instructions uses random parameters for operations, operands (and their addressing parameters), vector lengths, and vector lane assignments. To increase the code coverage, an increased range of executed instruction parameters is generated by random vector instruction sequences in repeated tests. For repeatability of certain tests and due to performance reasons of sequence generation, the random generation was done offline for the coverage evaluation and used a constant seed. However, this automatic random generation procedure of twin-based test sequences is also available as a single RISC-V program for later post-silicon validation or pre-silicon verification using an FPGA-based rapid prototyping system. The generated test sequences include input data (for the memories of the vector co-processor and the reference arrays), which is generated by the RISC-V.

After sequence generation, the vector instructions are executed on the V²PRO co-processor. The RISC-V core controls the execution of vector instructions on the vector co-processor, which includes the transfer of generated input data to the local memories and register files, the test instruction issue, and synchronization of the V²PRO. The RISC-V starts the vector instructions on the V²PRO with the corresponding parameters. After execution of the instruction sequence, the V²PRO is synchronized to ensure complete execution.

For the reference calculation, the vector instructions are calculated by the RISC-V reference model. This model processes the vector instructions sequentially. The reference calculation was implemented in C++ with the function behavior of each vector operation. The V²PRO array is processed by iterating through the vector co-processor's vector units and vector lanes. The internal memories of the vector co-processor (i.e., register files and local memories) are realized by integer arrays. A loop processes the vector operations with the defined complex addressing parameters

and implements the logic and arithmetic operations by corresponding bit-level exact masks (e.g., for bit shifting, or multiplication precision). The chaining mechanism of neighbor vector lanes is performed by sequential interleaving of vector element calculation and selection of chaining source data, simulating the vector co-processor architecture (as depicted in Fig. 3). The co-processor emulation produces functional correct results, however, it is a coarse-grained model, i.e., specific hardware implementation details are ignored (e.g., the vector lane's specific pipeline execution, or stall conditions).

After the execution of the test sequence on both twins, the results are compared to verify the correct execution. As the execution on the V²PRO creates result data inside the register file of the processing vector lanes, and the vector unit's local memory (e.g., by execution of store instructions), the content of these memories is copied to the external memory. These transfers are performed by the DMA units and copy the content of each memory to a reserved region of the external memory. After the DCMA finishes its flush to the external memory, the data is accessible for comparison by the RISC-V. In the final comparison, the stored results of vector co-processor execution and the reference memories are compared. When the compared result of the V²PRO and the RISC-V twin differs, the verification fails, and detailed results are reported (e.g., the executed instruction sequence, in- and output data, or resulting states of hardware registers).

V² PRO Extensions and Constraints

The vertical execution of vector instructions with complex addressing requires constraint checks during random vector instruction generation and register file addressing. One aspect includes the vector element's access order, defined by the operand addressing formula. During vertical (sequential) processing of vector elements in the vector lane pipeline, the source operands and the destination operand can generate hazards, like Read-After-Write (RAW) hazards. To reach high operating frequencies the hardware explicitly avoids control logic for hazard resolution. Therefore, the vector instructions are constrained to not include hazard addressing. Therefore, during random parameter generation, the vector operand addresses are checked to not generate RAW hazards for a specific period length (set to the pipeline depth).

Sequences of vector instructions may include the chaining concept of operand data. For example, when one vector lane requires chained data, the data-generating vector instruction is required before another vector instruction can be executed on the waiting vector lane. To avoid deadlocks, e.g., when all vector lanes wait for chained data no further vector instruction from the instruction FIFO gets issued, a corresponding constrain check is done in the generation procedure. During operand

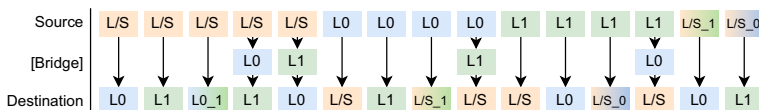


Fig. 8 Chaining combinations for the vector instructions in a vector unit with two processing vector lanes (L0 & L1) and one L/S lane. Both source operands can use chaining as an operand source on L0 and L1. The L/S lane has a single chain source during store operations. Chain data may be broadcast by use as source operand on different vector lanes

source selection, a chaining source can be randomly selected and require corresponding vector instructions on the selected source vector lane to generate the chained data at some time. The implementation of this constraint was achieved by tracking the individual chaining states of vector lanes during vector instruction generation.

At the end of sequence generation, specific vector instructions can be required to finish chain sequences. When a chain is present in the generated instruction sequence, e.g., by generation of an vector instruction with a chain input operand, a corresponding chain instruction on the selected source vector lane is required to generate those data. Therefore, the vector lane with chaining input operands awaits data and won't execute the following instructions until the source data are generated. Another example usage of chaining is the L/S lane to work as a chain source (load) or chain target (store). A processing vector lane can use chaining in both source operands and can generate chaining data. The resulting valid chaining combinations for vector instructions are shown in Fig. 8.

With the vertical execution of varying vector lengths, also multiple small-length vector instructions can be used as chain sources in one long vector instruction. The execution of vector instructions in between two chained vector instructions is possible in some cases (e.g., on a chaining uninvolved vector lane, or in a not blocked vector lane state). It is also possible to end chain source generating vector instructions and to interleave a different chain. Chained data can be used for broadcast as well. By the detailed chain state tracking during vector instruction generation, the vector lane's state is derived and the vector lane may get disabled as a target for new vector instructions. The chains are generated by a random selection of vector instruction parameters (e.g. chain source/target attribute) and added to the executed instruction sequence. Finalizing vector instructions to finish a test sequence are generated, based on the final chain state to ensure the valid execution of generated instruction sequences.

An architecture-specific extension for vector instruction generation focuses on the *instruction FIFO* in the vector units. To enable the filling of the instruction FIFO, at first a long executing vector instruction is prepended to a large sequence, to give the RISC-V sequence issue calls the required time to start vector instructions and to fill the FIFO memory. The generated number of vector instructions in a sequence gets configured in the framework, as the depth of the instruction FIFO of the vector co-processor can be configured.

Further parts of the vector co-processor are the special *configuration registers*. One *configuration register* configures the result shift in multiply vector instructions (see Table 3). Another *configuration register* sets the reset data source in multiply-and-accumulate vector instructions for the accumulation register reset (e.g., immediate, constant zero, or register file address). As the long vertical vectors are processed sequentially, another *configuration register* defines the reset condition of the accumulation register (e.g., reset once, or when the x-, y- or z-iterator increments). For coverage of the *configuration register* states and the correct functionality of the configured execution modes (i.e., shift amounts for multiplication, reset modes for the multiply-accumulate vector instruction, and reset value configurations for multiply-accumulate vector instructions), random configurations are emitted for each executed

test sequence. To cope with these features in the RISC-V twin, corresponding code snippets in the reference calculation were added.

The current framework uses the vector element addressing for a three-dimensional addressing formula of the V²PRO architecture. By its modular structure, the framework allows the extension for other addressing modes (e.g., other complex addressing definitions). Also, concepts like mixed vector mode (e.g., horizontal SIMD), or indirect address chaining methods can be integrated into the flow.

5 Evaluation Results

To evaluate the effectiveness of the proposed verification and validation methodology, code coverage metrics are measured through RTL simulation running with Questa Sim-64 2021.3. During simulation, the functional verification degree of a target processor architecture implementation is evaluated and can be used in later post-silicon validation to check the covered hardware units. The following coverage metrics show the scope of hardware evaluation of the executed applications.

- *Statement Coverage* The metric measures how many code lines of the (VHDL) hardware description file were executed. It ignores statically disabled statements as conditional generated hardware structures (`if <(false expression)>(generate)`).
- *Branch Coverage* This metric evaluates how many branch paths are executed, e.g., if a `if-then-` was taken.
- *Expression Coverage* This metric evaluates how many cases of an expression were evaluated in an assignment.
- *Condition Coverage* This metric provides coverage statistics on how many logical conditions are evaluated in conditional branches. Full condition coverage requires that each condition in a decision has taken on all possibilities.
- *FSM Coverage* (State & Transition): This metric measures how many FSM states (based on the signal definition of custom types) have been covered as well as how many state transitions occurred.
- *Toggle Coverage* For each signal, this metric counts the bit-wise toggles and reports on all the signals in the design that do not change.

The system of this paper is available by an FPGA implementation and allows the execution of the verification framework on a post-production scale. The FPGA, programmed by the generated bitstream, can introduce functional errors, e.g., when timing constraints haven't been met. For a slack-free version of the V²PRO system, the vector co-processor (which comprises 8 clusters with 8 vector units each) uses a 400 MHz clock. Additionally, the AXI interconnect and the RISC-V both use a 200 MHz clock. In this configuration, the verification framework test programs are executed on the same hardware descriptions that were evaluated during the simulation.

Table 4 Coverage Metrics for the handwritten Test Cases of RISC-V Compliance [26] for the Base Instruction Set, the M-Extension and Both

Total Assembly Instr		Statement Coverage	Branch Coverage	Expression Coverage	Condition Coverage	FSM States & Transitions	Toggle Coverage
RV32I	860,973	91.17 %	89.46 %	90.00 %	53.84 %	95.83 %, 81.81 %	75.68 %
RV32M	28,543	91.56 %	87.06 %	90.00 %	57.14 %	100.00 %, 84.09 %	82.79 %
RV32IM	889,516	99.40 %	99.07 %	98.33 %	79.12 %	100.00 %, 88.63 %	87.87 %
Total Bins: EIS-V	[#]	1012	515	76	94	30, 103	460

Bold shows value maximization

Table 5 Coverage Metrics for PATARA framework for different test methods

Test cases [# Total assembly Instr]	Statement Coverage	Branch Coverage	Expression Coverage	Condition Coverage	FSM States & Transitions	Toggle Cover- age
All Instruction	98.61 %	98.52 %	95.58 %	80.21 %	100.00 %, 88.63 %	83.37 %
Combinations [6,983]						
Instruction Combina- tion + operand switching (base) [9,722]	98.71 %	98.70 %	95.58 %	81.31 %	100.00 %, 94.28 %	83.53 %
base	98.80 %	98.89 %	95.58 %	81.31 %	100.00 %, 93.18 %	84.39 %
+ interleaving test [17,294]						
base + interleaving	99.50 %	99.81 %	97.05 %	96.70 %	100.00 %, 100.00 %	84.98 %
+ sequences [1,851,852]						
base + interleaving	100.00 %	100.00 %	100.00 %	100.00 %	100.00 %, 100.00 %	86.60 %
+ multiple sequences						
+ Cache tests [6,732,901]						
Total Bins: EIS-V [#]	1012	515	76	94	30, 103	460

Bold shows value maximization

5.1 RISC-V Processor Verification Results

During the design of the RISC-V VHDL processor implementation, the RISC-V compliance test suite [26] was used as a fast and reference test suite for functional verification. With this test suite no bugs were found, therefore the implementation passed this functional verification. However, not all VDHL code lines were covered during the simulations, indicating that neither all possible hazard conflicts were tested nor the implemented hardware architecture was optimally described in VHDL. The use of our proposed PATARA framework increased the thoroughness of the test cases, not

in the functionality but in the number of hazard conflicts, and it is also applicable for different pipeline length implementations. This allows, for example, to detect bugs in our custom 6-stage pipeline implementation. As a side effect of analyzing the code coverage reached by the test programs in simulation, it was possible to optimize the VHDL implementation, which is directly translated into a more compact and faster hardware implementation (i.e., for FPGA or ASIC design).

In the following, the PATARA framework is evaluated on a VHDL model of a 6-pipeline stage RV32IM processor and compared to the RISC-V compliance test suite [26]. It is worth mentioning that during the evaluation, RISC-V system class instructions were not executed, and therefore the coverage metrics do not evaluate the accesses to/from the control and status registers, neither for the handwritten test suite, nor for the PATARA framework generated test programs. The capability for interrupts or the sleep mode is also not tested by these test suites, and coverage was disabled for the related entities by using pragma directives inside the VHDL source code. In general, the RISC-V control and status registers, and the privileged architecture specifications require further verification methods shown in [6] and are out of the scope of this paper, which focuses on the unprivileged specification.

For the RV32IM instruction set in this paper, the handwritten RISC-V compliance test suite [26] consists of 17,765 concrete test instructions, which include the base instruction set and the M extension. The number of total executed instructions is higher than the number of test instructions, as test initialization adds further executed instructions. The accumulated coverage results for the handwritten tests are shown in Table 4.

It is worth mentioning that the toggle coverage is still quite low in comparison to the other metrics. This is mainly related to unchecked regions of the data and instruction address regions. The test programs did not use such large address spaces, especially for the instruction memory.

The coverage results for PATARA are shown in Table 5. For the PATARA framework, the **base** test case includes the tested RISC-V instructions, their variants, and the switching of the source operands. Adding the interleaving of instructions to the executed tests increases the coverage metrics. However, 100% coverage is not reached because not all forwarding mechanisms and cache mechanisms are tested.

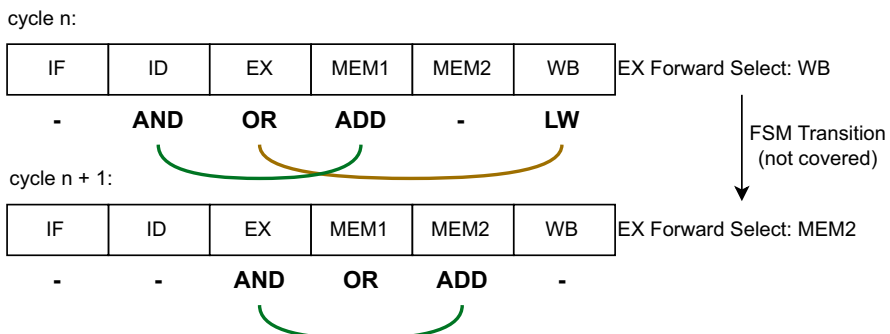


Fig. 9 Missed coverage for FSM Transition of hazard-based forwarding source from WB to MEM2 stage of the handwritten testbenches

Enabling all the capabilities of PATARA for testing increases the coverage metrics to 100%. The only exception to 100% coverage metrics is the toggle coverage, which can only reach up to 86.6%. The reasons for this are similar to the handwritten compliance tests.

5.2 Why does the Standard Verification not Reach Maximal Coverage Rates?

The handwritten test programs of the RISC-V compliance test suite reach high coverage, as shown in Table 4. However, these test programs do not trigger all the possible hazards, reducing the statement coverage to 99.4%. This is also translated into lower branch, condition, and FSM transition coverage metrics.

Not all transitions between consecutive forwarding path selections are used within the handwritten test cases, e.g., the EX stage forward selection source does not switch sequentially from WB to MEM2 and from WB to MEM1. An example of this is shown in Fig. 9. PATARA ensures the use of all possible forwarding paths and consecutive combinations for both source operands taking into account the pipeline depth during sequence generation.

During the evaluation of our PATARA framework, several bugs in the hardware description were found and fixed. For example, a concrete bug was found in the data forwarding unit, which did not correctly update the source of the forwarded operand when a multicycle division operation was not finished. As our multicycle division implements a variable number of cycles, this bug requires several test cases to cover all hazard and forwarding combinations. Another not covered hazard situation is related to the multiplication unit. This unit is split into two stages, EX and MEM1, and special considerations for the targeted RISC-V pipeline implementation should be taken in hardware when a data cache miss (e.g., from Load) is produced during a multicycle multiplication. Also, the data forwarding from a load instruction into the source of a JALR (Jump-And-Link-Return) instruction is another not detected hazard of the handwritten test suite. For most data hazard situations related to data load instructions, the required pipeline stall generation derived from a memory data cache miss is uncovered. Second, since instruction cache misses only rarely occur on small test cases, only a few tests, like the long branch test case, generate cache misses. As the branch test focuses on the branch instructions, other hazards in combination with those instruction cache misses are not covered. In general, several bugs related to a combination of cache misses and stall conditions were found and fixed.

Variations in the number of implemented pipeline stages require manually modifying the handwritten test cases to achieve full coverage of all hazards. This is resolved in the PATARA framework.

5.3 Which Coverage Metrics can be Improved with the REVERSI Approach of PATARA Compared to Standard Verification?

PATARA achieves full code coverage verification, due to the random generation of cache misses and forwarding hazard detection tests. The resulting code coverage of PATARA is shown for the case study of the 6-stage RISC-V implementation in Table 5. Long instruction sequences increase the statistical occurrence of different

Table 6 Coverage metrics of the system (excluding Memory Interface) for 84 handwritten test cases

Component(s)	Statement Coverage	Branch Coverage	Expression Coverage	Condition Coverage	FSM States & Transitions	Toggle Coverage
Risc-V	96.04 %	94.17 %	93.42 %	80.85 %	73.33 %, 38.83 %	72.62 %
V ² PRO	95.03 %	91.20 %	73.68 %	87.87 %	93.75 %, 90.09 %	79.83 %
→ Lane 0 (of Cluster 0, Unit 0)	97.43 %	93.55 %	96.29 %	72.00 %	100.00 %, 100.00 %	89.41 %
→ Lane 1 (of Cluster 0, Unit 0)	89.93 %	81.01 %	62.96 %	72.00 %	100.00 %, 100.00 %	77.57 %
System	93.38 %	90.06 %	77.02 %	77.57 %	80.00 %, 48.04 %	68.29 %
Total Bins: EIS-V [#]	1012	515	76	94	30, 103	460
Total Bins: V ² PRO [#]	1712	1149	95	165	16, 22	52241
Total Bins: System [#]	3403	2145	235	437	70, 179	59469

The V²PRO consists of 8 Vector Lanes in 2 Clusters, each has 2 Vector Units

kinds of data dependencies and allow PATARA to cover the missed hazard conditions of the handwritten test suite. By using the base instructions and reverse sequences, PATARA already achieves coverage metrics near the handwritten test suite with slightly increased condition coverage. The full coverage capability of the randomized method is shown by the extensive randomized test sequences with more than 626k instructions.

5.4 V²PRO Co-Processor Verification Results

For functional verification of the V²PRO co-processor, each test case is calculated on the RISC-V twin and the V²PRO co-processor itself. The results are compared and verified by the RISC-V core afterward. The code sections of the application that run on the co-processor (i.e., the V²PRO instruction) were executed and measured. The functional verification of the co-processor by different data sets and more random sequences was done by execution on a (faster) FPGA system as well.

The vector co-processor units and clusters work independently (e.g., vector instructions are broadcasted and executed in parallel on clusters and units) and therefore, do not increase the coverage metrics. For simulation time reduction of the time-consuming hardware simulation (for coverage extraction), the simulated number of vector clusters and units in the co-processor was reduced to one.

Handwritten tests give a reference test suite for the co-processor. These test cases utilize all vector operations but do not vary much regarding vector lengths, operand and encoding (e.g. register file source, immediate, or chaining), complex addressing parameters, and the combination of vector instructions in sequences. The tests include vector instructions independently at selected corner cases for operands and data. As shown in Table 6, the handwritten test cases utilize vector lane 0 for processing and achieve an expression coverage of only 73.68 % for the V²PRO co-processor. The goal of test case generation with full coverage for the co-processor requires test applications with more chaining combinations and further hardware condition checks, as required for complex addressing configurations and chaining. Also, the evenly distribution of operations to both processing vector lanes is desired for complete coverage.

Table 7 Coverage metrics of the co-processor (excluding memory interface) for randomly generated test cases

Test #	Unit(s)	Statement Coverage	Branch Coverage	Expression Coverage	Condition Coverage	FSM States & Transitions	Toggle Coverage
Random [1]	V ² PRO	92.11 %	87.11 %	51.57 %	87.27 %	100.00 %, 100.00 %	66.34 %
	→ Lane 0	86.93 %	78.30 %	22.22 %	76.00 %	100.00 %, 100.00 %	80.80 %
	→ Lane 1	87.15 %	78.64 %	22.22 %	76.00 %	100.00 %, 100.00 %	71.68 %
Random [10]	V ² PRO	99.06 %	97.82 %	56.84 %	98.18 %	100.00 %, 100.00 %	80.46 %
	→ Lane 0	97.64 %	96.27 %	25.92 %	96.00 %	100.00 %, 100.00 %	92.77 %
	→ Lane 1	98.92 %	97.28 %	33.33 %	100.00 %	100.00 %, 100.00 %	92.67 %
Random [100]	V ² PRO	100.00 %	99.04 %	91.57 %	98.78 %	100.00 %, 100.00 %	83.53 %
	→ Lane 0	100.00 %	98.98 %	92.59 %	100.00 %	100.00 %, 100.00 %	94.52 %
	→ Lane 1	100.00 %	98.98 %	88.88 %	100.00 %	100.00 %, 100.00 %	94.48 %
Random [1000]	V ² PRO	100.00 %	99.30 %	96.84 %	98.78 %	100.00 %, 100.00 %	86.07 %
	→ Lane 0	100.00 %	99.32 %	100.00 %	100.00 %	100.00 %, 100.00 %	95.23 %
	→ Lane 1	100.00 %	99.66 %	100.00 %	100.00 %	100.00 %, 100.00 %	95.23 %
Random [1000] + FIFO	V ² PRO	100.00 %	100.00 %	100.00 %	100.00 %	100.00 %, 100.00 %	86.21 %
	→ Lane 0	100.00 %	100.00 %	100.00 %	100.00 %	100.00 %, 100.00 %	95.23 %
+ Mod. Constraints	→ Lane 1	100.00 %	100.00 %	100.00 %	100.00 %	100.00 %, 100.00 %	95.23 %
Total Bins: V ² PRO	[#]	1712	1149	95	165	16, 22	52241

Bold shows value maximization

The number of executed sequences (each of length 7) is 1, 10, 100, and 1000. In the final case, additional sequences with modified constraints to random generation were added (higher probability for vectors of length 1, FIFO filling; see Sect. 4.4)

Table 7 shows the randomly generated test sequences' coverage metrics. As more sequences are executed, the random vector instruction generation creates more chaining combinations, as shown in Sect. 4.4. Randomized operand selection and variation in the parameters of complex addresses and immediate operands increase the code coverage. Both final rows include the massive generation of vector instructions with 1000 sequences.

The implemented constraints of vector instruction generation limit the generation process to valid vector instructions and non-locking vector chains. As the uniformly distributed random vector length generation ranges between one and the maximum

vector length, small vectors are underrepresented in the random generated sequences. To cover the not executed corner cases (e.g., chaining stall conditions in specific states), small vector lengths were enforced by a second constraint set. With sequences of small vector lengths, these code parts get covered as well. The instruction FIFO logic for full conditions was tested by further generated long sequences.

5.5 How can a Higher/Full Coverage Rate be Reached?

Full code coverage can be reached by the execution of test applications that require the execution of all implementation code lines, conditions, etc. For the hazard-related coverage metrics of the RISC-V implementation, this requires the test cases to trigger all hazards. Randomized vector instruction sequences lead to higher coverage as more code is evaluated, especially when constraints are used to trigger hazards. The number of required randomized vector instructions for full coverage is based on statistical factors. With increasing test sequence length, required by deep pipeline hardware implementations, also the execution time becomes relevant. The use of reference golden models (i.e., simulators) for the verification could be prohibitive due to the extra time required to generate the reference results. Self-testing methods as used by the PATARA framework avoid the use of such golden models, increasing the verification speed. A combination of handwritten test cases with random sequences can also be used for fast coverage maximization.

On the V²PRO co-processor, the generation of randomized vector instructions needs to cover a larger number of parameters to define the vector instructions. This increases the execution time for the high number of test sequences required to reach full coverage. The framework of this paper constrains the generation of vector instruction sequences by limiting the parameters to their valid range. In terms of toggle coverage, the execution of multiple random test data sets will cover more data values. With the twin-base approach, the executed test sequences can be repeatedly executed with different data on the system without needing to transfer the tested data for an external verification. The generation of data was achieved by a pseudo-random number generation by the RISC-V. For functional correctness, fast execution time, and high data coverage, the FPGA implementation was validated by repeated execution of the generated sequences with different data. The breakup of execution time for one random sequence with at least 7 random vector co-processor instructions shows an average of 86.04 ms for random sequence generation (when generated on the RISC-V), 2.68 ms for data generation (24576 Bytes² by a pseudo-random xoroshiro128plus software implementation on the RISC-V [5]), 0.05 ms for data transfers by DMA units, 0.01 ms for V²PRO execution, 16.10 ms for RISC-V-based reference calculation, and 6.76 ms for the RISC-V compare. When the complete verification framework's application is executed on the system (including time-consuming constraint checking during random test sequence generation), only the communication interface for error reporting is required for reporting (e.g., UART).

²The RISC-V creates data for one local memory: 8192 * 16-bit, and two vector register files: 1024 * 32-bit (each), resulting in 24576 Bytes of randomly generated data for each sequence.

5.6 What does 100% Coverage Mean for Hardware Verification?

A coverage report with 100% code coverage only indicates that all the code lines and conditional expressions of the hardware architecture description were executed during simulations. For a full validation against the RISC-V instruction set, several points of failure may still exist, as they may be masked on the generated self-test programs and unhandled by the target RISC-V hardware implementation. Therefore, even with complete code coverage, it cannot be guaranteed that a specific RISC-V implementation is completely correct.

The verification of the V²PRO co-processor depends on the previously verified RISC-V. A full coverage report verifies that all implemented lines of code were evaluated in the tests, leaving out code conditions to handle possible errors, not present in terms of hardware description. As the verification of V²PRO co-processor execution relies on a software twin on the RISC-V (implementation of a golden model), failures can be masked by erroneous execution on this model as well.

The PATARA framework includes the generation of REVERSI-based RISC-V tests with dependent instruction chains as introduced by the framework extensions in Sect. 4 in this paper. Furthermore, the address generation for memory access operations is randomized, to randomly generate data hazards related to cache misses. With this approach, the RISC-V hardware implementation characteristics are tested beside the functional correctness and do not require (manual) adaption to achieve 100 % code coverage for the custom hardware.

The self-testing test cases can be used for validation with no need for external data transfers and no external golden model execution as shown by the test application to verify the V²PRO co-processor with the RISC-V. On the utilized FPGA, long-running tests enable the coverage of large data ranges within random test sequences. The data sets are generated by the RISC-V and can be exchanged by different random functions easily. Further validation scenarios include the parallel execution of the presented framework on multiple systems. The scale of verification systems is performed easily, as no bottleneck for transfers of generated input data or output data for external verification exists. Architecture changes, e.g., ISA changes, can be added to the modular framework easily and the test scope gets extended to cover those changes.

6 Conclusions

With the REVERSI approach of the extended PATARA framework for test generation, functional verification of an implemented RISC-V hardware architecture reaches maximum code coverage. Moreover, when using randomized test programs, the execution time is not limited by a reference model, which is also interesting for post-silicon validation. In comparison to handwritten test frameworks, such as the RISC-V compliance test framework, complex combinations of instructions are generated by dynamic randomized instruction sequences. This allows to cover any implementation-dependent hazard condition and improves the code coverage metrics. For an exemplary RISC-V implementation, supporting the RV32IM instruction set and a

6 pipeline stage hardware architecture, the condition coverage can be increased from 79.12% with the handwritten test suite up to 100% by using the PATARA framework generated test programs. In addition to these coverage-maximizing randomized test programs, the reference result is generated and verified within the programs itself, based on the REVERSI method and without requiring any golden reference or external simulator. The core of the PATARA framework is independent of the target instruction set architecture and allows a flexible verification of any hardware implementation even on sophisticated hardware designs with deeper pipeline stages or multi-cycle units. The target instruction-set architectures are described with plug-ins. This allows the PATARA framework to be used for verification on different instruction-set architectures, as shown in this paper with the adoption of the RV32IM.

The verification of an attached co-processor for vector instruction execution is performed by the use of constrained random test sequence generation. The test generation covers all features of the vector co-processor and achieves 100 % coverage for the hardware implementation of the vector co-processor. With the RISC-V as a verification system, external communication for input or output data is not required and allows excessive self-testing on a system level by execution of test sequences on two paths of the twin-based verification approach; the RISC-V software twin and the V²PRO co-processor itself.

Acknowledgements This work was partly funded by the German Federal Ministry of Education and Research (BMBF) under project number 16ME0379 (ZuSe-KI-AVF).

Author Contributions S.G. wrote the main manuscript text and prepared figures. All authors reviewed the manuscript.

Funding Open Access funding enabled and organized by Projekt DEAL. This work was partly funded by the German Federal Ministry of Education and Research (BMBF) under project number 16ME0379 (ZuSe-KI-AVF).

Data Availability No datasets were generated or analysed during the current study.

Declarations

Conflict of interest The authors declare no conflict of interest.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

1. Ahmadi-Pour, S., Herdt, V., Drechsler, R.: Constrained random verification for RISC-V: Overview, evaluation and discussion. In: MBMV 2021; 24th Workshop. VDE (2021)
2. Alliance, C.: Random instruction generator for RISC-V processor verification. <https://github.com/google/riscv-dv> (2023)
3. AMD: AXI Verification IP (VIP). <https://www.xilinx.com/products/intellectual-property/axi-vip.html> (2024)
4. Austin, T.M.: DIVA: A dynamic approach to microprocessor verification. J. Instruction-Level Parallelism (2000)
5. Blackman, D., Vigna, S.: Scrambled linear pseudorandom number generators. ACM Trans. Math. Softw. (TOMS) **47**(4), 1–32 (2021)
6. Bruns, N., Herdt, V., Große, D., et al.: Toward RISC-V CSR compliance testing. IEEE Embedded Systems Letters (2021)
7. Campbell, B., Stark, I.: Randomised testing of a microprocessor model using SMT-solver state generation. Sci. Comput. Program. (2016)
8. Fournier, L., Asaf, S.: Fpgen—a deep-knowledge test generator for floating point verification. Tech. rep., IBM Research Report, Computer Science, H-0140 (2002)
9. Gautschi, M., Schiavone, P.D., Traber, A., et al.: Near-threshold RISC-V core with DSP extensions for scalable IoT endpoint devices. IEEE Trans on Very Large Scale Int (VLSI) Sys (2017)
10. Gesper, S., Stuckmann, F., Wöbbekind, L., et al.: PATARA: extension of a verification framework for RISC-V instruction set implementations. In: International Conference on Embedded Computer Systems. Springer (2023)
11. Grosso, M., Ravotto, D., Sanchez, E., et al.: Functional test generation for DMA controllers. In: 11th Latin American Test Workshop. IEEE (2010)
12. Grosso, M., Perez Holguin, W.J., Ravotto, D., et al.: Functional verification of DMA controllers. J. Electron. Test. (2011)
13. Herdt, V., Drechsler, R.: Advanced virtual prototyping for cyber-physical systems using RISC-V: Implementation, verification and challenges. Sci. China Inf. Sci. (2022)
14. Herdt, V., Große, D., Drechsler, R.: Closing the RISC-V compliance gap: Looking from the negative testing side. In: 57th ACM/IEEE Design Automation Conference (DAC). IEEE (2020)
15. Herdt, V., Große, D., Drechsler, R.: Towards specification and testing of RISC-V ISA compliance. In: Design, Automation & Test in Europe Conference & Exhibition (DATE). IEEE (2020)
16. Imperas: Imperas delivers highest quality RISC-V RV32I compliance test suites to implementers and adopters of RISC-V. <https://www.imperas.com/articles/imperas-delivers-highest-quality-risc-v-rv32-i-compliance-test-suites-to-implementers-and> (2019)
17. Imperas: Imperas RISC-V riscvOVPSim reference simulator and architectural validation tests. <https://www.ovpworld.org/riscvOVPSimPlus> (2022)
18. Josephson, D.: The good, the bad, and the ugly of silicon debug. In: Proc. of the 43rd annual Design Automation Conference. ACM (2006)
19. Kaliorakis, M., Psarakis, M., Foutris, N., et al.: Accelerated online error detection in many-core microprocessor architectures. In: 2014 IEEE 32nd VLSI Test Symposium (VTS). IEEE (2014)
20. Martignoni, L., Paleari, R., Roglia, G.F., et al.: Testing CPU emulators. In: Proc. of the 18th int. symp. on Software testing and analysis. ACM (2009)
21. MP, P.K., Panda, S.K.: Design and verification of DDR SDRAM memory controller using systemverilog for higher coverage. In: International Conference on Intelligent Computing and Control Systems (ICCS). IEEE (2019)
22. Nahir, A., Ziv, A., Galivanche, R., et al.: Bridging pre-silicon verification and post-silicon validation. In: Proceedings of the 47th Design Automation Conference. ACM (2010)
23. Nannipieri, P., Di Matteo, S., Crocetti, L., et al.: Cycle-accurate verification of the cryptographic co-processor for the european processor initiative. In: International Conference on Applications in Electronics Pervading Industry, Environment and Society. Springer (2023)
24. Nolting, S., Giesemann, F., Hartig, J., et al.: Application-specific soft-core vector processor for advanced driver assistance systems. In: 27th International Conference on Field Programmable Logic and Applications (FPL). IEEE (2017)
25. RISC-V: RISC-V Unit Tests. <https://github.com/riscv/riscv-tests> (2020)

26. RISC-V Foundation: RISC-V architecture test SIG. <https://github.com/riscv/riscv-compliance>, Access: 2020-08-20 (2020)
27. Rodrigues, C.: A case study for formal verification of a timing co-processor. In: 10th Latin American Test Workshop. IEEE (2009)
28. Schiavone, P.D., Sanchez, E., Ruospo, A., et al.: An open-source verification framework for open-source cores: A RISC-V case study. In: 2018 IFIP/IEEE Int. Conf. on Very Large Scale Integration (VLSI-SoC). IEEE (2018)
29. Stuckmann, F., Fistanto, P.A., Payá-Vayá, G.: PATARA: A REVERSI-based open-source tool for post-silicon validation of processor cores. In: 2021 10th International Conference on Modern Circuits and Systems Technologies (MOCASST). IEEE (2021)
30. Thieu, G.B., Gesper, S., Payá-Vayá, G., et al.: ZuSE Ki-Avf: Application-Specific AI processor for intelligent sensor signal processing in autonomous driving. In: 2023 Design, Automation & Test in Europe Conference & Exhibition (DATE). IEEE (2023)
31. Wagner, I., Bertacco, V.: Reversi: Post-silicon validation system for modern microprocessors. In: IEEE International Conference on Computer Design. IEEE (2008)
32. Wagner, I., Bertacco, V.: Post-silicon and runtime verification for modern processors. Springer Science & Business Media (2010)
33. Waterman, A., Asanovic, K.: The RISC-V instruction set manual; Volume I: Unprivileged ISA (2019)
34. Waterman, A., Lee, Y., Patterson, D.A., et al.: The RISC-V instruction set manual, volume I: Base User-level ISA. EECS Department, UC Berkeley, Tech Rep UCB/EECS-2011-62 (2011)
35. Waterman, A., Lee, Y., Patterson, D.A., et al.: The RISC-V instruction set manual. Volume 1: User-level IS, version 2.0. Tech. rep., California Univ Berkeley Dept of Electrical Engineering and Computer Sciences (2014)
36. Zulberti, L., Di Matteo, S., Nannipieri, P., et al.: A script-based cycle-true verification framework to speed-up hardware and software co-design: Performance evaluation on ecc accelerator use-case. Electronics (2022)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.